

Evaluating Low-Shot Procedural Skill Transfer in Language-Agent Debugging

Jiachen Sun[✉]

ShanghaiTech University

Shanghai, China

13673527519sjc@gmail.com

Abstract

We study whether a natural-language SKILL.md induced from three debugging trajectories transfers to held-out language-agent debugging tasks. Our protocol records repair success, diagnosis-edit-test cycles, tokens, failed patches, and memory-attributed negative transfer. In a single-execution pilot on six verified PyBugHive black tasks, Auto SKILL.md ties a generic checklist and Reflexion at 5/6 solved. Task-level comparisons show lower process cost than reflection baselines but do not establish a stable advantage over the checklist. One negative-transfer case and an outlier-sensitive structural control illustrate why memory evaluation needs uncertainty, process, and misapplication measures rather than solve rate alone.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Intelligent agents*.

Keywords

language agents, software debugging, procedural memory, skill induction, negative transfer

ACM Reference Format:

Jiachen Sun. 2026. Evaluating Low-Shot Procedural Skill Transfer in Language-Agent Debugging. In *Proceedings of the 1st International Workshop on Agentic AI for Next-Generation Software Development (AgenticDev '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3843282.3844421>

1 Motivation and Protocol

Language-agent memory ranges from verbal reflection to distilled guidelines, workflows, and procedural repositories [2, 5, 6, 8]. In debugging, however, experience can accelerate a related repair or misdirect the agent toward an adjacent invariant. We therefore evaluate not only whether a memory solves a task, but also how it changes repair cost and whether the trajectory supports memory-attributed negative transfer. This complements recent work that reports steps or token cost and observes harm from noisy memory [4].

We induce one fixed artifact from $K = 3$ redacted training trajectories. The induction contract forbids task-specific names and patches and requires three parts: applicability triggers, an ordered debugging procedure, and failure modes. The artifact is deployed

unchanged on held-out tasks. Five primary conditions share the same model, task prompts, isolated workspaces, tests, temperature, and budgets: Auto SKILL.md, a human-written generic checklist, Reflexion, length-matched Reflexion, and no memory. The checklist already contains reproduction, localization, minimal editing, and broad validation; Auto SKILL.md additionally encodes applicability exclusions, expected-versus-actual invariant compression, and failed-rerun classification.

We evaluate six reproducible PyBugHive black formatter bugs [1]. Each task/method pair has one execution. Metrics are computed from validated trajectories and test reruns. Negative transfer follows a predefined taxonomy but was assigned by one annotator. For task-level uncertainty, we use Wilson intervals for solve proportions, paired task-resampling bootstrap intervals on common-solved tasks, medians, and leave-one-task-out sensitivity. These do not estimate run-to-run variance.

2 Results and Robustness

Table 1 shows no solve-rate dominance: Auto SKILL.md, the checklist, and Reflexion each solve 5/6. Wilson 95% intervals are 0.44–0.97 for 5/6 and 0.30–0.90 for 4/6. On four common-solved tasks, Auto-minus-baseline mean token differences are –22.8k for Reflexion, –27.3k for length-matched Reflexion, and –11.8k for the checklist. Exploratory task-bootstrap intervals are –44.4 to –1.3k, –41.4 to –13.1k, and –30.3 to 7.1k, respectively. Cycle differences are –1.00, –1.00, and 0.00; the checklist interval (–0.75 to 0.75) again shows no separation. Thus the pilot differentiates Auto SKILL.md more clearly from reflection memories than from competent human-written advice.

The Auto SKILL.md token median is 52.6k, compared with 56.0k for the checklist and 68.5k for Reflexion; its leave-one-solved-task-out mean spans 48.0–62.7k. One Auto run on black_132 is labeled negative transfer: the first skill-guided patch increased full-suite failures from four to six. This label is useful diagnostic evidence, not a population-rate estimate.

A supplementary gpt-5.5 structure control preserves skill content but shuffles its organization. Both versions solve 6/6; Auto SKILL.md uses fewer cycles (1.67 vs. 2.17) and failed patches (0.50 vs. 0.83). Mean tokens favor shuffled content (102.0k vs. 112.0k), but medians are nearly identical (86.5k vs. 86.4k), and excluding one outlier reverses the mean ordering. The control therefore suggests process differences without showing that the canonical structure is necessary for solvability.



This work is licensed under a Creative Commons Attribution 4.0 International License. AgenticDev '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2985-0/2026/10

<https://doi.org/10.1145/3843282.3844421>

Table 1: Primary same-model pilot. Tokens are solve-conditional means. NT is the number of memory-attributed negative-transfer flags.

Method	Solved	Cycles/solved	Tokens/solved	Failed/run	NT
Auto SKILL.md	5/6	2.00	56,867	0.83	1
Generic checklist	5/6	2.00	71,694	1.33	0
Reflexion memory	5/6	2.60	65,915	1.33	0
Length-matched Reflexion	4/6	2.75	75,231	1.50	0
No memory	4/6	3.00	92,031	1.17	0

3 Interpretation and Threats

The induced artifact is inspectable: relative to the checklist, it makes applicability boundaries, invariant compression, and failed-rerun categories explicit. Yet these differences are not equivalent to demonstrated new knowledge. The tie in solve rate and cycles, together with a checklist token interval that crosses zero, leaves open whether trajectory induction adds more than competent human-written procedure. Content-matched section ablations are needed to identify causal components.

The evidence also has three direct validity limits. First, six selected tasks and one execution per condition cannot characterize stochastic agent behavior; our intervals describe task variation only. Second, all primary tasks come from one formatter project, so within-family transfer may not extend to repository-wide debugging benchmarks such as SWE-bench [3, 7]. Third, cycle records compress internal search, token means are outlier-sensitive, and one annotator assigned negative transfer. Repeated seeds, direct tool instrumentation, and independent annotation are prerequisites for stronger performance claims.

4 Conclusion

The main contribution is an inspectable protocol for evaluating low-shot procedural-memory transfer, including process cost and misapplication. The pilot provides initial process evidence but does not establish stable superiority. Code, artifacts, and validated trajectories are available at <https://github.com/damogu123/trace2skill>.

References

- [1] Gábor Antal, Norbert Vándor, István Kolláth, Balázs Mosolygó, Péter Hegedűs, and Rudolf Ferenc. 2024. PyBugHive: A Comprehensive Database of Manually Validated, Reproducible Python Bugs. *IEEE Access* 12 (2024), 123739–123756. doi:10.1109/ACCESS.2024.3449106
- [2] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2026. MEMP: Exploring Agent Procedural Memory. In *Findings of the Association for Computational Linguistics: ACL 2026*. Forthcoming. arXiv:2508.06433 <https://arxiv.org/abs/2508.06433>
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*. International Conference on Learning Representations. <https://openreview.net/forum?id=VTF8yNQm66>

- [4] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=jL7fwchScm>
- [5] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., Red Hook, NY, USA. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- [6] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning*. <https://openreview.net/forum?id=NTAhi2JEEE>
- [7] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA. https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [8] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. AAAI Press, 19632–19642. doi:10.1609/aaai.v38i17.29936

Received 2026-07-10; accepted 2026-08-20

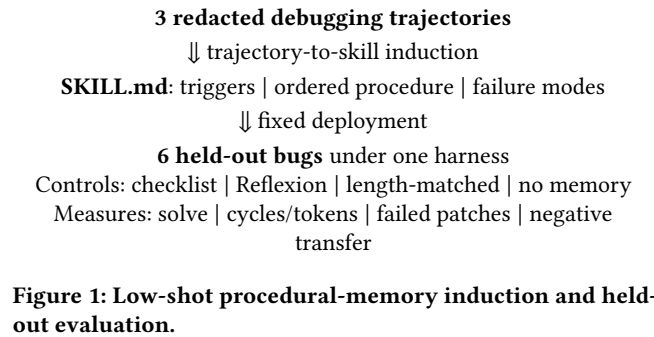


Figure 1: Low-shot procedural-memory induction and held-out evaluation.